CTRE: Team Walkthrough

A guide for explaining the Cyber Threat Reconstruction Engine as a group. The project is split into four parts, one per presenter. Each part has the points to make, a short script, and the questions you are likely to get.

Presenters and their parts:

- **Vivek**: the problem and the big picture (Part 1)
- **Kshitij**: disk forensics: `$MFT`, `$UsnJrnl`, and timestomping (Part 2)
- **Moksh**: network metadata and correlation (Part 3)
- **Aranck**: the interface, testing, safety, and results (Part 4)

Suggested order: Vivek sets up the problem and the architecture, Kshitij and Moksh explain the two halves of the analysis, and Aranck shows it running and reports what it found. The live demo at the end is driven by one person while the others answer questions.

The one-line version (anyone can say this)

CTRE reads three kinds of Windows evidence offline, finds files whose timestamps were faked, ties data leaving the machine to files changing on disk, and puts it all on one timeline. It never decrypts traffic.

Part 1. Vivek: the problem and the big picture

What you are explaining

Why the project exists, and how the whole system fits together. Point at Figure 1 in the paper (the pipeline diagram) while you talk.

The points to make

- A Windows break-in leaves traces in several places at once: a file lands on disk, the filesystem logs the change, and data leaves over the network.
- These traces are normally looked at with separate tools, so the link between them gets lost. A file appearing and a transfer leaving a few seconds later are only suspicious together.
- CTRE takes three artifacts (the `$MFT`, the change journal, and a packet capture), reads them together, and answers two questions: were any file timestamps faked, and did data leave the machine right after a file changed?
- Everything is offline and read-only. The tool opens copies of already-collected evidence and never writes to them.
- One deliberate choice: CTRE does not decrypt network traffic. It uses only what is already visible in a capture, such as who was contacted, when, and how much data moved.

Short script

"When a machine is compromised, the evidence is scattered. The file system knows a new file appeared, the change journal knows exactly when, and the network capture knows that something was sent out. Each tool sees one piece. Our project reads all three together. The pipeline is on this diagram: the three artifacts on the left go through read-only parsers, then into two analysis stages, one for faked timestamps and one for data exfiltration, and the results come out as a single timeline and a report."

If asked

- *Why offline?* Because incident response works on copies of collected evidence, not on the live

machine. It also means the tool can run on an isolated analysis box with no risk to the original.
- *Why not just use one existing tool?* The point of the project is the link between artifacts, which single-artifact tools do not show.

Part 2. Kshitij: disk forensics and timestomping

What you are explaining

How CTRE reads the two NTFS artifacts and how it catches faked timestamps. Point at Figure 2 (the decision flowchart) for the detection rules.

Background you may need to give

- The `$MFT` (master file table) has one record per file. Each record stores timestamps in two places: `$STANDARD_INFORMATION` (SI), which is what tools and users see and is easy to rewrite, and `$FILE_NAME` (FN), which the kernel maintains and is harder to forge.
- The `$UsnJrnl` is the change journal: an append-only log the kernel keeps of every filesystem change, with the time and the kind of change.
- Timestomping is an anti-forensic trick: an attacker rewrites a malicious file's timestamps so it looks old and blends in with system files.

The points to make

- The parser reads each `$MFT` record, applies the NTFS fixups (a check NTFS uses to catch torn writes), and pulls out both timestamp sets.
- The journal parser handles real-world size: an extracted journal can be several gigabytes of mostly empty space, so it memory-maps the file and skips the empty regions instead of loading it all. A one-gigabyte sparse test file parses in about 4.5 seconds.
- Detection works by disagreement. The four rules are: - `SI_BEFORE_FN` (high): the file's SI creation time is earlier than its FN creation time. A file cannot exist before its own name record. - `SI_BEFORE_JRNL` (high): the SI creation time is earlier than the journal's record of the file being created. - `ZERO_SUBSECOND` (medium): several SI timestamps have their sub-second digits zeroed, which many timestomping tools produce. - `NO_JRNL_FOR_CHANGE` (medium): the SI modified time is newer than any journal entry for the file.
- Each finding comes with a severity and a written reason, so it reads as an explanation, not just a flag.

The part worth emphasizing (the false-positive guards)

Two of these rules would misfire on real disks without care, and we fixed that:

- The journal is a ring buffer, so its oldest surviving record is usually not the file's creation. The `SI_BEFORE_JRNL` rule only fires when the journal actually holds a creation record, not just an old edit.
- Windows reuses an `$MFT` record number after a file is deleted, so journal history from a deleted file can attach to the file that replaced it. The detector also matches on the record's sequence number, which tells the two apart.

Both of these have their own regression tests.

Short script

"Windows keeps two sets of timestamps. One is easy to fake, one is not. Our detector plays them against each other and against the change journal. If the easy-to-fake time says a file was created in 2019 but the journal says the file was actually created last week, that gap is the tampering. This flowchart is the exact set of checks, and each one produces a severity and a plain-English reason."

If asked

- *Can this false-positive?* The sub-second rule can: files from ZIP archives or FAT media also have zeroed sub-seconds, so we treat that as a medium-level lead, not proof. The two high-severity rules have guards for journal wrap and record reuse.
- *What if the journal is huge?* It is memory-mapped and the empty space is skipped, so size is not a problem.

Part 3. Moksh: network metadata and correlation

What you are explaining

How CTRE reads the packet capture without decrypting it, and how it links network activity to disk activity. Point at Figure 3 (the correlation flow).

The points to make

- The pcap reader parses the classic capture format with Ethernet and IPv4. It builds one flow record per connection: the endpoints, ports, how many packets and bytes went each way, and the first and last time it was seen.
- It also reads two things that are in the clear even on encrypted connections: DNS query names, and the server name a client sends when starting a TLS connection (the SNI).
- It never decrypts anything. For the exfiltration question that is fine, because the signal is the size, timing, and destination of a transfer, not the bytes inside it.
- The correlator looks for a large outbound flow whose start time is within a set window of a disk change in the journal. That pairing is the shape of staging a file and then sending it out.
- Everything, disk events and network events, goes onto one timeline sorted by time, so the sequence of create, log, and send reads top to bottom.

Why keyless analysis is the right call here

The original brief included reading encryption keys out of memory and decrypting the traffic. We left that out on purpose: the same code decrypts anyone's traffic, so it is a general-purpose interception tool regardless of how it is packaged. The exfiltration signal does not need it. This is a good point to make because it shows a deliberate security and ethics decision, not a gap.

Short script

"We do not break encryption. We do not need to. A capture still tells us who was contacted, when, and how much left the machine, plus the hostname in the DNS query and the TLS handshake. Our correlator takes every large outbound transfer and checks whether a file changed on disk right around the same time. When those two line up, that is the pattern of data being staged and exfiltrated, and it goes on the timeline next to the disk events."

If asked

- *How do you know a flow is outbound and large?* We compare byte counts per direction and apply a size threshold, which is adjustable in the UI.
- *Could this miss slow exfiltration?* Yes, a transfer spread thinly over a long time can stay under the threshold and window; that is a known limitation.

Part 4. Aranck: interface, testing, safety, and results

What you are explaining

How someone actually uses the tool, how we know it works, the safety of the local server, and what it found on the test case.

The points to make (interface)

- Two ways to run it: a command line, and a local web page that opens in the browser when you run the tool with no arguments.
- The web page explains the project as you go: an overview with the pipeline diagram, numbered steps, a built-in sample case, and a reference section with the rules and a glossary. After a run it prints a plain summary of what it found.

The points to make (safety of the local server)

- The server listens only on the local machine (the loopback address).
- It checks the `Host` header, so a different site cannot reach it by name.
- It requires a custom header on every API call, which stops other websites open in the browser from driving it.
- It sends a strict Content-Security-Policy, and it deletes uploaded files when it stops.
- All evidence text is put on the page as plain text, so a hostile file name cannot inject code into the page or the report.

The points to make (testing)

- There is a generator that writes fake artifacts in the exact on-disk formats the parsers read, so the generator and the parsers check each other.
- It plants one timestomped file and one timed transfer, so there is a known right answer.
- There are 20 tests covering the timestamp math, each parser, the four rules, the correlation, the report escaping, and a full upload-run-download cycle against the live server. All pass.
- The pcap reader was cross-checked against scapy, a separate packet library, and the packet count, DNS name, TLS server name, and byte totals matched.

The points to make (results on the sample)

- Parsed 2 `$MFT` entries, 4 journal records, 122 packets in 2 flows.
- Flagged the planted file `svchost.exe` with three rules: two high (`SI_BEFORE_FN`, `SI_BEFORE_JRNL`) and one medium (`ZERO_SUBSECOND`).
- The benign file was not flagged.
- Linked the outbound transfer of about 160 KB to the disk change for that file.
- The timeline shows the story: `svchost.exe` claims a 2019 creation in the `$MFT`, the journal records its real creation in 2026, then a DNS lookup, a TLS handshake, and the transfer to the external host.

Short script

"You run it and this page opens. You can load the sample case with one click and hit run. Here is what it found: it caught the file we planted, `svchost.exe`, with three separate rules, it left the normal file alone, and it tied the outbound transfer to the moment that file changed. The timeline reads like a story: the file claims to be from 2019, the journal says it is actually new, and then the data leaves. Everything you see here is backed by 20 passing tests, and we checked our network parser against a separate library to be sure the numbers are real."

If asked

- *Is the local web server a security risk?* No, it is loopback-only and has several guards against other

sites reaching it; details above.
- *How confident are you in the results?* The pipeline is proven on controlled data, not on a real intrusion. The tests show it works end to end and fires on a known case; real-world detection rates would need captured cases to measure.

The live demo (one driver, the rest answer)

1. Run `python -m ctre` to open the web page.
2. Click **Load sample case**, then **Run analysis**.
3. Read the **What the engine found** summary out loud.
4. Scroll to the **findings** (point out the two high-severity rules on

`svchost.exe`), then the **correlation**, then the **timeline**. 5. Toggle the timeline source filters to show disk versus network events. 6. Click **Download HTML report** to show the shareable output.

If anything fails live, fall back to the screenshots and the report PDF.

Quick reference

- `$MFT`: NTFS master file table; one record per file, with its timestamps.
- `$UsnJrnl:$J`: NTFS change journal; append-only log of filesystem changes.
- SI vs FN: `$STANDARD_INFORMATION` (easy to fake) vs `$FILE_NAME` (harder).
- Timestomping: faking a file's timestamps to hide when it was made or changed.
- Flow: one network connection summarized by endpoints, ports, timing, bytes.
- SNI: the server hostname sent in the clear in a TLS handshake.
- Exfiltration: moving data off the system.

Commands:

- `python -m ctre`: open the web UI.
- `python -m ctre gen sample_artifacts`: write the sample artifacts.
- `python -m pytest -q`: run the test suite.